

Unit 4 – Failure Handling (Simplified Detailed Notes)

These notes explain every topic and subtopic from Unit 4 in a simple, detailed, and exam-friendly manner. Difficult concepts are broken down using easy language, real-life examples, and step-by-step explanations.

Introduction to Failure Handling

Distributed systems are made up of many computers working together. Because many components are involved, failures are common.

One important feature of distributed systems is partial failure.

What is Partial Failure?

A partial failure happens when:

- Some parts of the system fail
- Other parts continue working normally

Example:

Suppose a food delivery app uses:

- Database server
- Payment server
- Tracking server

If the payment server crashes but tracking still works, this is a partial failure.

Goal of Failure Handling:

The main aim is to design systems that:

- Continue working despite failures
- Recover automatically
- Reduce performance impact

Fault Tolerance

Fault tolerance means the system continues providing services even when faults occur.

A fault-tolerant system:

- Detects failures
- Handles failures
- Continues operation

Example:

Google servers continue working even if one data center fails because other servers take over.

Fault tolerance is closely related to dependability.

Dependability means:

“How trustworthy and reliable the system is.”

Dependability Requirements

A dependable system should satisfy several properties.

1. Availability

Availability means the system is ready to use whenever users need it.

Example:

ATM machines should be available 24/7.

A highly available system:

- Rarely goes offline
- Quickly recovers from downtime

2. Reliability

Reliability means the system works continuously without failure for a long time.

Difference Between Availability and Reliability:

- Availability focuses on a specific moment.
- Reliability focuses on a time duration.

Example:

A server that crashes for 1 millisecond every hour:

- Highly available
- But not highly reliable

Another example:

A server shut down for maintenance every August:

- Reliable during the year
- But lower availability

3. Safety

Safety means failures should not cause disasters.

Example:

In nuclear plants or aircraft systems:

- Even a small failure can be dangerous

The system must avoid catastrophic outcomes.

4. Maintainability

Maintainability means:

“How easily can the system be repaired?”

A maintainable system:

- Detects problems quickly
- Repairs faults easily
- Supports automatic recovery

5. Security

Dependable systems also require security.

This protects:

- Data integrity
- Confidential information
- System resources

Failure, Error and Fault

These three terms are closely related but different.

1. Fault

A fault is the root cause of a problem.

Examples:

- Programming bug
- Hardware damage
- Loose cable

2. Error

An error is an incorrect internal state caused by a fault.

Example:

A wrong calculation due to buggy code.

3. Failure

Failure happens when the system provides incorrect service.

Example:

Application crashes or gives wrong output.

Relationship:

Fault → Error → Failure

Real-Life Example:

- Fault = cracked brake pipe
- Error = brake pressure becomes incorrect
- Failure = car brakes stop working

Types of Faults

1. Transient Fault

- Happens once
- Disappears automatically

Example:

Temporary network disturbance.

If operation is retried, it usually succeeds.

2. Intermittent Fault

- Comes and goes repeatedly
- Difficult to detect

Example:

Loose electrical connection.

The system may work fine during testing and fail later.

3. Permanent Fault

- Continues until repaired

Examples:

- Damaged hardware
- Permanent software bug

Failure Models

Failure models help classify different kinds of failures.

Why are failure models important?

They help developers:

- Understand failures
- Design recovery mechanisms
- Build fault-tolerant systems

Failures may occur in:

- Servers
- Communication channels
- Both

Example:

Disk failure → file server affected → database affected

Types of Failures

1. Crash Failure

The server suddenly stops working.

Characteristics:

- Worked correctly before crash
- Produces no output afterward

Example:

Computer suddenly shutting down.

Key Idea:

“Stops completely.”

2. Omission Failure

The server fails to send or receive messages.

Types:

a) Receive Omission Failure

Server does not receive incoming messages.

Causes:

- Network issue
- No listening thread

b) Send Omission Failure

Server processes request but response is not sent.

Causes:

- Buffer overflow
- Communication failure

Problem:

Client may retry and create inconsistency.

3. Timing Failure

Response arrives at wrong time.

Cases:

- Too early
- Too late

Example:

Video streaming packets arriving too late cause lag.

4. Response Failure

Server responds incorrectly.

Types:

a) Value Failure

Wrong output value.

Example:

Calculator showing wrong answer.

b) State Transition Failure

System behaves unexpectedly.

Example:

ATM dispensing money without proper request.

5. Arbitrary (Byzantine) Failure

Most dangerous failure type.

The server may:

- Send random outputs
- Behave maliciously
- Send different answers to different users

Example:

A hacked server sending fake banking data.

Key Idea:

“Anything can happen.”

Failure Masking by Redundancy

Failure masking means hiding failures from users so the system still works.

Main Technique: Redundancy

Redundancy means adding extra resources.

Types:

1. Information Redundancy

Extra bits are added to detect and correct errors.

Example:

Hamming codes in communication systems.

2. Time Redundancy

Retry failed operations.

Examples:

- Resending messages
- Retrying transactions

Useful for temporary faults.

3. Physical Redundancy

Uses extra hardware or software.

Examples:

- Backup servers
- Process replication
- Multiple copies of data

Advantage:

If one component fails, another continues.

Process Resilience

Process resilience means the system survives process failures.

Idea:

Use multiple identical processes grouped together.

If one process fails:

- Another process takes over

Important Features:

- Groups can be created dynamically
- Processes can join or leave
- A process may belong to multiple groups

To users:

The whole group behaves like one process.

Example:

Cloud systems use many replicated servers.

Users do not notice when one server fails.

Types of Groups

1. Flat Group

All members are equal.

Characteristics:

- Decisions made collectively
- No coordinator
- No single point of failure

Advantages:

- Better fault tolerance

Disadvantages:

- Complex coordination

2. Hierarchical Group

One process acts as coordinator.

Characteristics:

- Coordinator controls decisions
- Easier management

Advantages:

- Simpler decision making

Disadvantages:

- Coordinator failure affects group

Failure Masking and Replication

Replication means creating multiple copies of processes.

Why replication?

A single process is vulnerable.

Multiple copies increase reliability.

k-Fault Tolerance:

A system is k-fault tolerant if it survives failure of k processes.

Replication Approaches:

1. Primary-Based Replication

Also called Primary-Backup Protocol.

Structure:

- One primary server
- Multiple backups

Working:

- Primary handles operations
- Backups stay updated
- If primary fails → backup becomes new primary

Example:

Database master-slave replication.

2. Replicated-Write Protocols

All replicas are equal.

Examples:

- Active replication
- Quorum systems

Advantages:

- No single point of failure

Disadvantages:

- Coordination becomes difficult

Fault Tolerance Level

Case 1: Crash Failures

Faulty servers stop silently.

Required servers = $k + 1$

Example:

To tolerate 2 failures:

Need 3 servers.

Case 2: Byzantine Failures

Faulty servers may send incorrect data.

Required servers = $2k + 1$

Example:

To tolerate 2 Byzantine failures:

Need 5 correct servers.

Important Byzantine Rule:

Total processes needed = $3k + 1$

Example:

If $k = 1$:

Need 4 total processes.

Agreement in Faulty Systems

Distributed systems often need consensus.

Consensus means:

“All nonfaulty processes agree on the same value.”

Example:

Bank servers agreeing on account balance.

Classic Example:

Byzantine Generals Problem.

Imagine:

Several army generals surround a city.

Some generals are traitors.

Goal:

All loyal generals must agree on:

- Attack

OR

- Retreat

Problem:

Traitors may send false information.

This is the Byzantine Agreement Problem.

Byzantine Agreement Problem

Assumptions:

- Processes are synchronous
- Messages arrive in same order
- Communication delay is bounded

Working:

1. Every process sends its value to others.
2. Processes exchange received information.
3. Majority voting is used.
4. Final agreement is reached.

Key Formula:

To tolerate k faulty processes:

Need at least $3k + 1$ total processes.

Example:

For 1 faulty process:

Need 4 total processes.

Reason:

Majority voting must overpower faulty nodes.

Consensus with Crash Failures

Consensus becomes difficult when processes crash.

Without failures:

Consensus is easy.

With failures:

Some processes may receive messages while others do not.

This creates inconsistency.

Goal:

Eventually all nonfaulty processes agree.

Flooding-Based Consensus

Flooding-based consensus spreads information to all processes.

Working:

1. Every process broadcasts its commands.
2. All processes merge received lists.
3. Same deterministic algorithm selects command.

Without failures:

All processes have same information.

With failures:

Some processes may miss data.

Solution:

- Wait for next round
- Gather more information
- Detect failed processes

Advantages:

- Eventually reaches agreement
- Works under crash failures

Handling Message Loss:

- Messages retransmitted
- Other processes rebroadcast messages

Raft Consensus Protocol

Raft is a practical consensus protocol.

Purpose:

Keep replicated servers consistent.

Structure:

- One leader
- Multiple followers

Leader:
Controls operations.

Followers:
Copy leader's log.

How Raft Works:

1. Client sends request to leader.
2. Leader adds operation to log.
3. Leader sends updates to followers.
4. Followers acknowledge.
5. Majority agreement commits operation.

Advantages:

- Simpler than Paxos
- Easy to understand
- Fault tolerant

Example:

Google-style distributed storage systems use similar ideas.

Leader Failure Handling in Raft

If the leader crashes:

- Followers elect new leader.

Requirements for new leader:

- Most updated log
- Latest committed operations

Why?

To prevent data loss.

Example:

Suppose operation o1 was committed before crash.

New leader must contain o1.

Clients resend requests if no response arrives.

Log Synchronization

Logs must remain identical across servers.

Leader ensures:

- Missing entries are copied
- Incorrect entries corrected

This maintains consistency.

Example:

If one follower missed updates during network failure:
Leader resynchronizes it later.

Failure Detection

Failure detection helps identify failed processes.

Methods:

1. Active Detection

Processes send heartbeat or ping messages.

Example:

“Are you alive?” packets.

2. Passive Detection

Processes wait for normal communication.

Timeout Mechanism:

If no reply arrives within time:
Process considered failed.

Problems:

- Network delay may cause false detection
- Packet loss may create confusion

Advanced Failure Detection

1. Gossip-Based Detection

Processes share information with neighbors.

Information spreads like gossip.

If updates about a node stop:
Node is considered failed.

Advantages:

- Scalable
- Distributed

2. Detection as Side Effect

Failures detected during normal communication.

Example:

If a node suddenly stops replying during data exchange.

Distinguishing Failure Types

Main challenge:

Did the process fail?

OR

Did the network fail?

Solution:

Use cooperative checking.

Example:

If node A suspects node B failed:

- Ask other nodes
- If nobody can reach B → likely failed

Failure Notification

Once failure is detected:

All nonfaulty processes must be informed.

Example: FUSE System

Working:

- Processes connected using spanning tree
- Nodes send ping messages

If one node stops responding:

- Neighbor suspects failure
- Information spreads recursively

Result:

Entire group learns about failure.

Advantage:

TCP reduces false failure detection due to network issues.

Exam Preparation Tip: Do not memorize only definitions. Focus on understanding the flow of failures, how systems recover, and why redundancy and consensus are important. Using real-life examples like banking systems, cloud servers, and mobile apps makes these concepts much easier to remember.